

Alvo da Auditoria:	Centralizador Lanches OP
Escopo do Projeto:	Código-Fonte Flask, SSH Tunnels, Nginx Config & Deploy
Auditor Responsável:	Arquiteto de Application Security (AI Agent)
Data de Emissão:	18 de Julho de 2026
Classificação:	CONFIDENCIAL / RESTRITO

Sumário

- 1. Resumo Executivo 3
- 2. Escopo da Auditoria 3
- 3. Metodologia de Segurança 4
- 4. Arquitetura Geral Analisada 4
- 5. Mapeamento da Superfície de Ataque 5
- 6. Relatório Completo de Vulnerabilidades 6
- 7. Matriz de Classificação de Riscos 10
- 8. Estatísticas e Métricas de Severidade 11
- 9. Componentes Avaliados como Seguros 11
- 10. Plano de Remediação e Ação Priorizado 12
- 11. Checklist de Hardening de Produção 13
- 12. Conclusão da Postura de Segurança 14

1. Resumo Executivo

Esta auditoria estática defensiva realizou um exame rigoroso do ecossistema do **Centralizador Lanches OP**, envolvendo o código-fonte desenvolvido em Python/Flask, os módulos javascript de pré-visualização, os scripts utilitários de túneis SSH persistentes e a configuração de servidores web Nginx ativos em produção.

Durante a análise, identificou-se um perfil de risco moderadamente elevado antes das remediações, caracterizado por falhas latentes de controle de acesso horizontal (IDOR/BOLA), desvios nos canais de webhook financeiros e falta de blindagem em uploads de arquivos. Com a aplicação imediata das defesas propostas e o desenvolvimento da suíte de segurança, a postura geral da aplicação foi substancialmente elevada, embora pontos operacionais de infraestrutura (como SSH e cabeçalhos HTTP) ainda necessitem de atenção.

2. Escopo da Auditoria

A auditoria de código-fonte contemplou os seguintes ativos do projeto:

- **[Código de Aplicação]** Arquivos Flask (``app.py``, ``auth.py``, ``models.py`` e blueprints ``routes_*.py``).
- **[Infraestrutura Local]** Robô dispatch e SSH tunnels (``local_whatsapp_dispatcher.py``, ``run_tunnel.py``).
- **[Servidor Web]** Arquivo de configuração ativo do Nginx no servidor EC2 (``lanches.conf``).
- **[Canais Externos]** Estruturação do canal de recepção de Webhook financeiro (``routes_webhook.py``).

3. Metodologia de Segurança

A auditoria seguiu estritamente uma abordagem estática defensiva (**Threat Modeling** e **Static Application Security Testing - SAST**), mapeando os componentes de software contra os controles do ****OWASP Top 10**** e os requisitos de verificação do ****OWASP ASVS v4.0****.

A análise classificou as severidades de risco multiplicando a probabilidade de exploração pelo impacto gerado ao negócio, gerando faixas de classificação padronizadas em Baixa, Média, Alta e Crítica.

4. Arquitetura Geral Analisada

A aplicação opera sob uma arquitetura híbrida distribuída:

- **Servidor de Produção (AWS EC2):** Hospeda o aplicativo web Flask sob servidor de aplicação Gunicorn e proxy reverso Nginx.
- **Banco de Dados Remoto (MariaDB):** Armazena dados de ordens de vendas de forma unificada e é blindado contra acessos externos diretos.
- **Estação Operacional Local:** Executa um script utilitário (``run_tunnel.py``) que abre uma conexão SSH reversa com a EC2 permitindo que o despachador local de mensagens (``local_whatsapp_dispatcher.py``) consulte a base de dados via porta local (3307) para acionar o robô de disparo via navegador Playwright.

5. Mapeamento da Superfície de Ataque

Identificamos quatro superfícies de ataque principais expostas no ecossistema:

1. **Interface do Administrador (Admin/Master Dashboard):** Exposta a acessos indevidos por falhas de multitenancy (IDOR) e ataques CSRF nas ações de configuração de templates e gerenciamento de ordens.
2. **Canal de Integração do Webhook (/webhook/efi):** Exposto diretamente na internet para recebimento de eventos do banco. A ausência de autenticação do remetente permitia a falsificação completa de transações.
3. **Mecanismo de Carregamento de Arquivos (/efi-config/save):** Permite upload de certificados, expondo a aplicação a riscos de armazenamento e execução de arquivos maliciosos caso a extensão não fosse filtrada.
4. **Túnel SSH de Banco de Dados:** A infraestrutura local confia na autenticidade do host remoto. O uso de opções que pulam a validação da chave do host expõe o túnel a sequestros de sessões por interceptação (Man-in-the-Middle).

6. Relatório Completo de Vulnerabilidades

ID: SEC-001	Vulnerabilidade: Broken Object-Level Authorization (IDOR) em Detalhes e Ações de Pedidos	
Categoria: OWASP A01:2021-Broken Access Control	Local: routes_admin.py (rotas /order//update_status, /order//details, /order//delete)	
Severidade: Alta	Probabilidade: Alta Impacto: Alta	
Explicação Técnica:	A aplicação carregava registros do banco usando o ID de forma direta nas rotas sem checar se a ordem consultada pertencia ao subsite de controle do administrador ativo.	
Impacto Potencial:	Vazamento total de relatórios de clientes de franquias concorrentes e alteração indevida de dados financeiros/pedidos de terceiros.	
Evidências:	Order.query.get_or_404(order_id) executado imediatamente sem verificação de propriedade.	
Correção Recomendada:	Aplicar validação cruzada do subsite do pedido contra o subsite associado à sessão do administrador logado.	
Exemplo de Implementação:	<pre>order = Order.query.get_or_404(order_id) if order.subsite_id != get_current_admin_subsite_id(): abort(403)</pre>	
Referências:	OWASP Top 10 (A01), CWE-639 (BOLA/IDOR), Requisito ASVS 4.0.3	

ID: SEC-002	Vulnerabilidade: Bypass de Autenticação e Confirmação de Transações no Webhook Financeiro	
Categoria: OWASP A07:2021-Identification and Authentication Failures	Local: routes_webhook.py (rota /webhook/efi)	
Severidade: Crítica	Probabilidade: Alta Impacto: Crítica	

Explicação Técnica:	O webhook de recebimento de confirmações de PIX aceitava requisições sem validar assinatura digital, chaves de acesso ou IP de origem, aprovando pagamentos com base no txid informado.
Impacto Potencial:	Bypass de cobrança. Um atacante poderia mandar pacotes POST falsificados aprovando qualquer pedido de forma gratuita.
Evidências:	Processamento imediato do txid enviado no corpo da requisição sem cabeçalhos de autenticação ou segurança mTLS.
Correção Recomendada:	Implementar restrição de acesso por meio de whitelist de endereços IP oficiais do gateway de pagamentos (EFI).
Exemplo de Implementação:	<pre>client_ip = request.headers.get('X-Forwarded-For', request.remote_addr) if client_ip not in EFI_WEBHOOK_IPS: abort(403)</pre>
Referências:	OWASP Top 10 (A07), CWE-287 (Improper Authentication), Requisito ASVS 3.1

ID: SEC-003	Vulnerabilidade: IDOR no Controle e Remoção de Itens do Catálogo
Categoria: OWASP A01:2021-Broken Access Control	Local: routes_admin.py (rota /item//delete)
Severidade: Alta	Probabilidade: Média Impacto: Alta
Explicação Técnica:	A rota permitia a deleção de itens do cardápio recebendo apenas o id do item por parâmetro POST sem checar se a loja dona do item pertencia ao subsite do administrador autenticado.
Impacto Potencial:	Negação de serviço em catálogos de terceiros, permitindo que administradores concorrentes deletem produtos alheios.
Evidências:	Falta de checagem do item_store.subsite_id comparado ao subsite do usuário ativo.
Correção Recomendada:	Realizar checagem de escopo de subsite através da loja associada ao item antes de comitar a deleção.
Exemplo de Implementação:	<pre>item_store = Store.query.get(item.store_id) if not item_store or item_store.subsite_id != subsite_id: abort(403)</pre>
Referências:	OWASP Top 10 (A01), CWE-285 (Improper Authorization)

ID: SEC-004	Vulnerabilidade: Falta de Proteção contra Solicitações Forjadas Cross-Site (CSRF)
Categoria: OWASP A01:2021-Broken Access Control	Local: Global (Blueprints admin_bp e master_bp em routes_admin.py e routes_master.py)
Severidade: Alta	Probabilidade: Alta Impacto: Alta
Explicação Técnica:	A aplicação não utilizava tokens anti-CSRF nos formulários POST administrativos, permitindo submissões silenciosas iniciadas de outros sites a partir do navegador da vítima.
Impacto Potencial:	Um invasor poderia fazer o administrador executar ações de exclusão ou alteração de configurações forçando cliques em páginas falsas na internet.

Evidências:	Falta de integração e validação de tokens nos formulários e nas funções after/before_request.
Correção Recomendada:	Configurar o cabeçalho SameSite nos cookies e criar um interceptor de segurança para checar a procedência do Referer/Origin nas chamadas POST.
Exemplo de Implementação:	<pre>app.config['SESSION_COOKIE_SAMESITE'] = 'Lax' # No before_request: if parsed_origin.netloc != parsed_host.netloc: abort(400)</pre>
Referências:	OWASP Top 10 (A01), CWE-352 (Cross-Site Request Forgery), Requisito ASVS 4.2

ID: SEC-005	Vulnerabilidade: Upload de Certificado Sem Validação de Tipo de Arquivo
Categoria: OWASP A01:2021-Broken Access Control	Local: routes_admin.py (rota /efi-config/save)
Severidade: Média	Probabilidade: Média Impacto: Alta
Explicação Técnica:	O upload de certificado de banco para a API da EFÍ usava secure_filename mas não restringia quais extensões de arquivo eram gravadas no diretório /certs.
Impacto Potencial:	Upload de arquivos executáveis maliciosos ou scripts no servidor, criando potenciais vetores de execução remota de código.
Evidências:	Salvamento em disco imediato do efi_cert_file sem whitelist de extensões.
Correção Recomendada:	Garantir que apenas extensões de certificados públicos/privados (.pem ou .crt) sejam aceitas.
Exemplo de Implementação:	<pre>ext = os.path.splitext(filename)[1].lower() if ext not in ['.pem', '.crt']: abort(400)</pre>
Referências:	OWASP Top 10 (A01), CWE-434 (Unrestricted Upload of File)

ID: SEC-006	Vulnerabilidade: Ausência de Cabeçalhos de Segurança HTTP no Proxy Reverso Nginx
Categoria: OWASP A05:2021-Security Misconfiguration	Local: Nginx config (/etc/nginx/conf.d/lanches.conf)
Severidade: Média	Probabilidade: Alta Impacto: Média
Explicação Técnica:	A configuração do servidor web Nginx não envia cabeçalhos de segurança essenciais como Strict-Transport-Security (HSTS), X-Frame-Options ou X-Content-Type-Options.
Impacto Potencial:	Exposição de administradores e clientes a sequestro de tráfego HTTP, ataques de Clickjacking ou farejamento indevido de tipo MIME de arquivos estáticos.
Evidências:	Ausência de diretivas add_header no arquivo lanches.conf.
Correção Recomendada:	Adicionar as diretivas de cabeçalhos de segurança na configuração do server block do Nginx.
Exemplo de Implementação:	<pre>add_header X-Frame-Options "SAMEORIGIN" always; add_header X-Content-Type-Options "nosniff" always;</pre>
Referências:	OWASP Top 10 (A05), CWE-693 (Protection Mechanism Failure)

ID: SEC-007	Vulnerabilidade: Bypass de Validação de Host SSH nos Scripts de Túnel e Deploy	
Categoria: OWASP A05:2021-Security Misconfiguration	Local: run_tunnel.py e deploy_now.py	
Severidade: Média	Probabilidade: Média Impacto: Alta	
Explicação Técnica:	Os scripts locais executam o comando ssh com o parâmetro -o StrictHostKeyChecking=no, ignorando a verificação de autenticidade da chave criptográfica do host remoto.	
Impacto Potencial:	Ataques de Man-in-the-Middle (MitM). Se o canal DNS ou rede local for sequestrado, as credenciais ou a chave SSH podem ser vazadas para uma máquina invasora mascarada.	
Evidências:	Parâmetro -o StrictHostKeyChecking=no configurado no array do comando subprocess.Popen.	
Correção Recomendada:	Configurar o arquivo local de hosts conhecidos (known_hosts) e remover a diretiva que desativa o StrictHostKeyChecking.	
Exemplo de Implementação:	<pre>cmd = ['ssh', '-i', key_path, '-N', '-L', '3307:127.0.0.1:3306', 'ec2-user@danieldmello.store']</pre>	
Referências:	OWASP A05:2021, CWE-321 (Use of Hard-coded Cryptographic Key)	

ID: SEC-008	Vulnerabilidade: Exposição de Arquivos Temporários de Edição (Swap Files) no Servidor	
Categoria: OWASP A05:2021-Security Misconfiguration	Local: Diretório do Nginx (/etc/nginx/conf.d/)	
Severidade: Baixa	Probabilidade: Média Impacto: Baixa	
Explicação Técnica:	Presença de arquivos residuais de editores de texto (.espetaria.conf.swp e .lanches.conf.swp) no diretório de configuração do Nginx.	
Impacto Potencial:	Vazamento de informações de configuração do servidor ou histórico de edições do administrador caso o usuário tenha privilégios limitados de leitura em pastas do sistema.	
Evidências:	Presença de .lanches.conf.swp e .espetaria.conf.swp listados no diretório ativo.	
Correção Recomendada:	Efetuar a limpeza de arquivos swap orfãos e configurar o editor para não gerar arquivos residuais ou limpá-los automaticamente.	
Exemplo de Implementação:	<pre>sudo rm -f /etc/nginx/conf.d/*.swp</pre>	
Referências:	OWASP A05:2021, CWE-527 (Recovery of Temporary Files)	

7. Matriz de Classificação de Riscos (Matriz de Riscos)

A classificação de gravidade dos riscos mapeados é resumida na seguinte relação de Probabilidade × Impacto:

ID	Probabilidade	Impacto	Risco Final
SEC-001	Alta	Alta	Alto
SEC-002	Alta	Crítica	Crítico
SEC-003	Média	Alta	Alto
SEC-004	Alta	Alta	Alto
SEC-005	Média	Alta	Médio
SEC-006	Alta	Média	Médio
SEC-007	Média	Alta	Médio
SEC-008	Média	Baixa	Baixo

8. Estatísticas e Métricas de Severidade

A distribuição das vulnerabilidades por criticidade é apresentada a seguir:

- **Crítica:** 1 (12.5%)
- **Alta:** 3 (37.5%)
- **Média:** 3 (37.5%)
- **Baixa:** 1 (12.5%)

Total de achados mapeados: 8

9. Componentes Avaliados como Seguros

Sanitização de Imagens: O upload de fotos de perfil do usuário (`upload_attachment`) foi revisado e é considerado altamente robusto. A rotina verifica o tipo MIME, valida a estrutura física da imagem usando Pillow (`PIL.verify()`) e regenera os nomes dos arquivos via UUID4. Essa arquitetura neutraliza eficientemente a execução de scripts embutidos nos metadados de fotos de perfil ou Path Traversal.

Segurança das Chaves do Banco de Dados: As variáveis sensíveis locais e as credenciais reais são carregadas em tempo de execução exclusivamente a partir de variáveis de ambiente (`os.getenv()`), evitando que chaves de API ou senhas fiquem estaticamente gravadas e visíveis no histórico de commits do repositório Git.

10. Plano de Remediação e Ação Priorizado

Recomendamos o seguinte plano de execução priorizado para saneamento completo da aplicação:

- **Prioridade 1 (Crítica e Altas - Correções Aplicadas):** Implantar as validações de IDOR nas rotas de pedidos, ativar a proteção de Origem/Referer (CSRF) e a filtragem de IPs no webhook. (Já implantados com a suíte de testes unitários).
- **Prioridade 2 (Média - Infraestrutura):** Adicionar cabeçalhos de segurança na configuração do Nginx (lanches.conf), exigir a verificação de chaves de host em conexões SSH e restringir uploads de certificados para .pem/.crt.
- **Prioridade 3 (Baixa - Higiene):** Excluir arquivos de swap residuais no diretório /etc/nginx/conf.d/ do servidor.

11. Checklist de Hardening de Produção

- ☒ Validar e comitar o helper de subsite de rotas administrativas.
- ☒ Rejeitar conexões POST sem headers de origem corretos.
- ☒ Restringir acesso do webhook `/webhook/efi` aos IPs oficiais do banco.
- ☐ Remover StrictHostKeyChecking=no de deploy_now.py e run_tunnel.py.
- ☐ Adicionar HSTS, X-Frame-Options e nosniff no Nginx.
- ☐ Limpar arquivos swap (.swp) na pasta de configurações do Nginx.

12. Conclusão da Postura de Segurança

O ecossistema do Centralizador Lanches OP deu passos significativos em direção a um ambiente robusto e defensável com a introdução de controles estáticos contra IDOR, proteção de origens CSRF e barreira de IPs no webhook PIX.

A aplicação da nova suíte de testes de regressão de segurança garante que melhorias futuras não reintroduzam falhas. Concluindo, a postura atual do projeto é considerada ****Adequada****, dependendo apenas da aplicação dos patches de infraestrutura no Nginx e SSH recomendados neste relatório para atingir um nível de blindagem completo e maduro.